CS-590 Database Design and Development

5-1 Assignment: Document Databases and MongoDB

Malgorzata Debska

Southern New Hampshire University

Zach Probst

March 2, 2026

The purpose of this assignment was to simulate a real-world migration scenario in which the Child Nutrition database, originally developed in PostgreSQL for the U.S. Department of Agriculture, was evaluated for potential implementation in MongoDB. This process involved setting up a MongoDB environment, performing extract-transform-load (ETL) operations on two datasets (Nutrient Descriptions and Nutrient Values), executing queries, and comparing MongoDB's structure and querying style to PostgreSQL.

MongoDB User Creation and Database Instantiation

A new user was created in the MongoDB instance using the createUser() command within the admin database. The user was granted read and write privileges specifically for the child_nutrition database. Creating a dedicated user aligns with best practices in database security by ensuring role-based access control rather than relying on unrestricted administrative access. The MongoDB database was instantiated using the use of child_nutrition command in the Mongo shell (mongosh). Unlike relational systems, MongoDB does not require an explicit database to be created. The database is automatically created once data is inserted into a collection.

ETL Process (Extract, Transform, Load)

The ETL process was performed on the following datasets provided in Codio:

- Module_Five_Nut_Descs.json
- Module_Five_Nut_Vals.json

Extract

The datasets were accessed from the Assignment Resources folder in Codio. These files stood for the Nutrient Description and Nutrient Values tables previously structured in PostgreSQL.

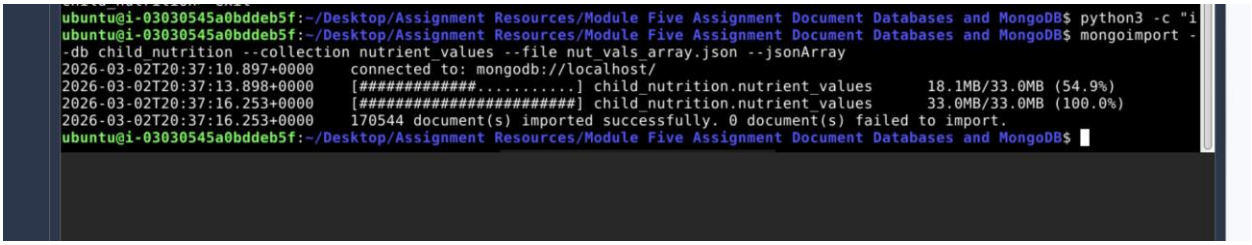
Transform

The original JSON files had wrapper objects such as "NUTVAL" and "NUTDESC" that enclosed arrays of records. MongoDB's mongoimport tool requires a properly formatted JSON array when using the --jsonArray flag. Therefore, the data was transformed by extracting only the internal arrays and saving them into new JSON files. Also, validation was performed to ensure the JSON format was correct before importing. This step was necessary because improperly formatted JSON causes import errors. This transformation step shows how document databases may require structural adjustments when migrating from relational datasets.

Load

The cleaned JSON arrays were imported into MongoDB using the mongoimport command:

mongoimport --db child_nutrition --collection nutrient_values --file nut_vals_array.json --jsonArray . The import process successfully inserted thousands of documents into the nutrient_values collection. The same process was repeated for the nutrient_descriptions collection.



Datatypes and Record Structure

MongoDB automatically decides field datatypes based on the JSON input. After import, the following datatypes were seen:

- "Cn code" – String
- "Nutrient code" – String
- "Nutrient value" – String
- "Per unit" – String
- "Date added" – String

- _id – ObjectId (automatically generated)

These datatypes are expected because the source JSON stored values in quotation marks. MongoDB does not enforce a predefined schema unless explicitly configured, so it stores data exactly as provided. MongoDB breaks data into separate records by storing each object within the JSON array as an individual document inside a collection. Unlike relational systems, MongoDB does not need foreign key relationships between tables. Each document can have all related information, allowing data to be retrieved without performing joins.

Query Creation and Execution

After importing, queries were executed in mongosh to confirm successful data access. The following commands were used:

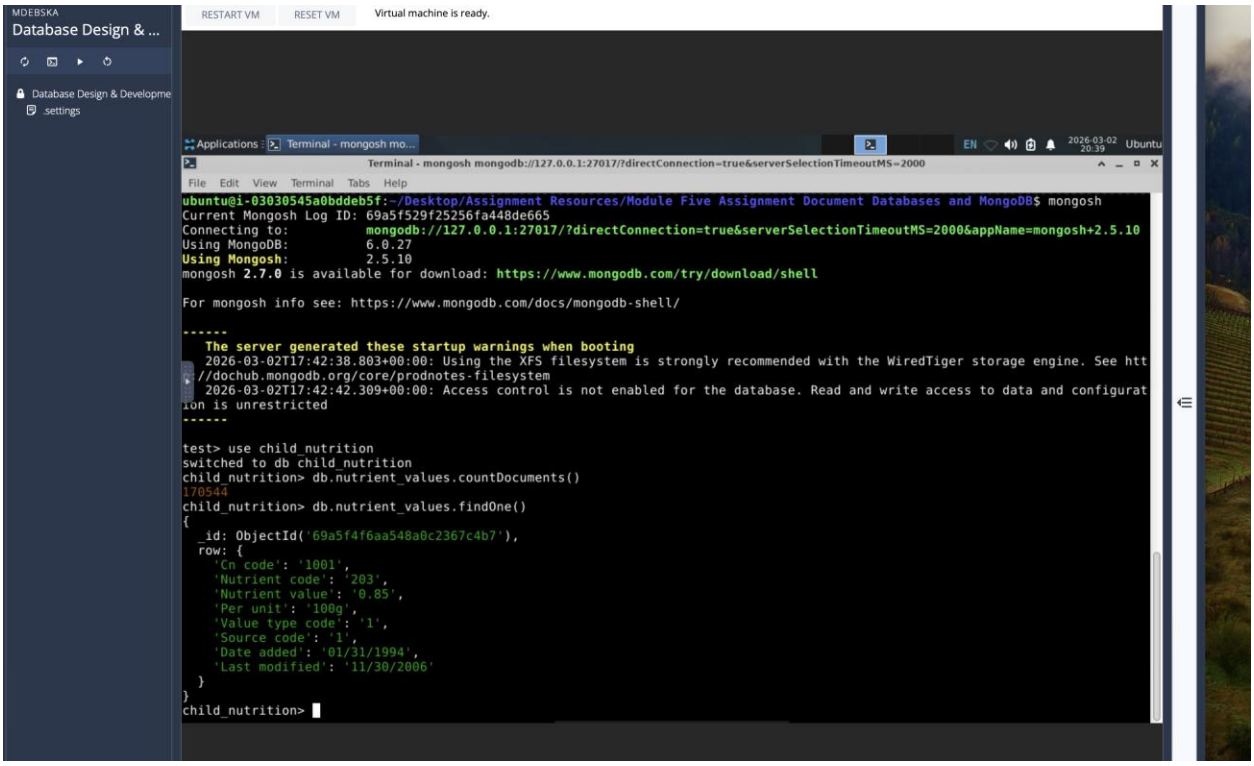
```
db.nutrient_values.countDocuments()
db.nutrient_values.findOne()
```

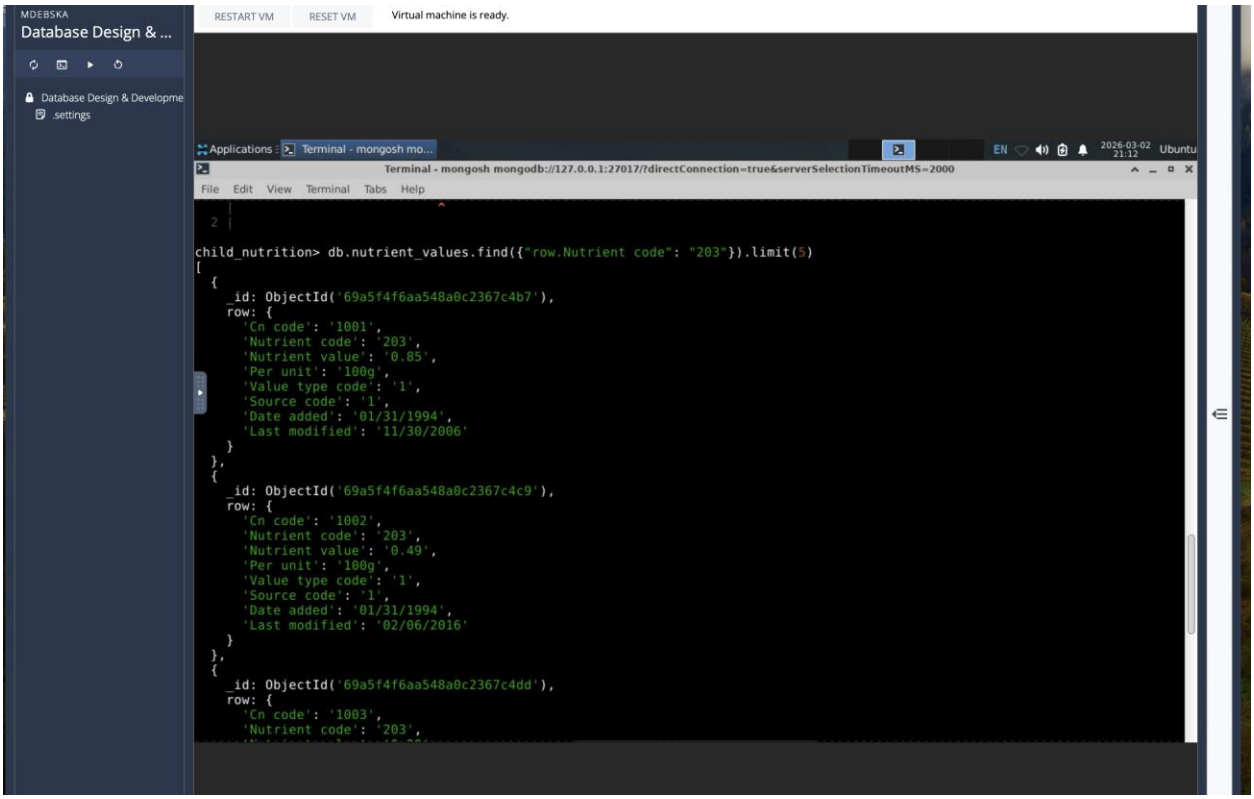
The countDocuments() command confirmed that records were successfully inserted into the collection. The findOne()command returned a complete nutrient document, verifying accessibility.

A filtered query was also executed:

```
db.nutrient_values.find(
  {"row.Nutrient code": "203" }
).limit(5)
```

This query proves MongoDB's use of dot notation to access nested fields within documents.


MDEBSKA
Database Design & ...
Database Design & Developme
.settings
RESTART VM
RESET VM
Virtual machine is ready.
Applications
Terminal - mongosh mo...
EN
2026-03-02
20:39
Ubuntu
Terminal - mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
File Edit View Terminal Tabs Help
ubuntu@i-03030545a0bddeb5f:~/Desktop/Assignment Resources/Module Five Assignment Document Databases and MongoDB$ mongosh
Current Mongosh Log ID: 69a5f529f25256fa448de665
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.10
Using MongoDB: 6.0.27
Using Mongosh: 2.5.10
mongosh 2.7.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
The server generated these startup warnings when booting
2026-03-02T17:42:38.803+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See htt
//dochub.mongodb.org/core/prodnotes-filesystem
2026-03-02T17:42:42.309+00:00: Access control is not enabled for the database. Read and write access to data and configurat
ion is unrestricted
test> use child_nutrition
switched to db child_nutrition
child_nutrition> db.nutrient_values.countDocuments()
170544
child_nutrition> db.nutrient_values.findOne()
{
_id: ObjectId('69a5f4f6aa548a0c2367c4b7'),
row: {
'Cn code': '1001',
'Nutrient code': '203',
'Nutrient value': '0.85',
'Per unit': '100g',
'Value type code': '1',
'Source code': '1',
'Date added': '01/31/1994',
'Last modified': '11/30/2006'
}
}
child_nutrition>



Comparison: MongoDB vs. PostgreSQL

Data Model

PostgreSQL uses a relational model composed of tables, rows, columns, primary keys, and foreign key constraints. Relationships between Nutrient Descriptions and Nutrient Values are supported through structured links. MongoDB uses a document-based model where data is stored

as JSON-like documents within collections. Related data can be embedded within the same document rather than distributed across separate tables.

Schema Flexibility

PostgreSQL enforces a strict schema, requiring defined column types and relationships before inserting data. Structural changes require schema migrations. MongoDB offers flexible schema designs. Documents within a collection can vary in structure. This flexibility is helpful when handling evolving or semi-structured datasets.

Scalability

PostgreSQL traditionally scales vertically, meaning performance improvements require more hardware resources. MongoDB is designed for horizontal scaling through sharding, making it more suitable for distributed systems and large-scale applications.

Querying Style

PostgreSQL uses Structured Query Language (SQL), including SELECT statements and JOIN operations. MongoDB uses JavaScript-like syntax with commands such as find(), insertOne(), and updateOne(). Instead of joins, MongoDB relies on embedding or referencing documents.

Recommendation

Based on this evaluation, PostgreSQL is still the best solution for the Child Nutrition database. The dataset is highly structured, relational, and requires consistency and integrity across multiple

related entities. Government systems, particularly those run by the U.S. Department of Agriculture, require strict data validation and reliable transactional support, both of which are strengths of relational databases. MongoDB would be helpful in scenarios involving frequent schema changes, unstructured data ingestion, or large-scale horizontal distribution. However, for a structured nutrition database requiring relational consistency, PostgreSQL is the proper choice.

This assignment proved the process of migrating structured relational data into a document-based database. Creating a MongoDB user, instantiating a database, performing ETL operations, importing JSON datasets, finding datatypes, and executing queries, the practical differences between MongoDB and PostgreSQL were explored. While MongoDB provides flexibility and scalability, PostgreSQL offers stronger schema enforcement and relational integrity. For the Child Nutrition database, PostgreSQL is still the recommended solution due to its structured nature and regulatory requirements.